MTE Jailbreak Detection Developer's Guide

MTE Version 2.1

Table of Contents

1 Introduction
2 Terms and Abbreviations
3 Prerequisites
4 Caveats
5 Architecture
6 SDK Structure
7 Options
8 Status
9 Basic C API
- 9.1 Overview
- 9.2 Files
- 9.3 Functions
  - 9.3.1 Nonce Mutator
    - 9.3.1.1 `mte_jail_n_cb_android_arm32_d` (C)
    - 9.3.1.2 `mte_jail_n_cb_android_arm64_d` (C)
    - 9.3.1.3 `mte_jail_n_cb_android_x86_s` (C)
    - 9.3.1.4 `mte_jail_n_cb_android_x86_64_s` (C)
    - 9.3.1.5 `mte_jail_n_cb_ios_arm64_d` (C)
    - 9.3.1.6 `mte_jail_n_cb_ios_x86_64_s` (C)

10 C Support API
11 C++ Language Interface
- 11.1 Overview
- 11.2 Files
- 11.3 Types
  - 11.3.1 Enumerations
    - 11.3.1.1 `MteJail::Algo` (CPP)
      - 11.3.1.1.1 `MteJail::Algo::aNone` (CPP)
      - 11.3.1.1.2 `MteJail::Algo::aAndroidArm32Dev` (CPP)
      - 11.3.1.1.3 `MteJail::Algo::aAndroidArm64Dev` (CPP)
      - 11.3.1.1.4 `MteJail::Algo::aAndroidX86Sim` (CPP)
      - 11.3.1.1.5 `MteJail::Algo::aAndroidX86_64Sim` (CPP)
      - 11.3.1.1.6 `MteJail::Algo::aIosArm64Dev` (CPP)
      - 11.3.1.1.7 `MteJail::Algo::aIosX86_64Sim` (CPP)
  - 11.3.2 Classes
    - 11.3.2.1 `MteJail`
- 11.4 Functions
  - 11.4.1 Jailbreak
    - 11.4.1.1 `MteJail` Constructor (CPP)
    - 11.4.1.2 `MteJail` Destructor (CPP)
    - 11.4.1.3 `MteJail::setAlgo` (CPP)
    - 11.4.1.4 `MteJail::setNonceSeed` (CPP)

11.4.1.5 `MteJail::nonceCallback` (CPP)

12 C# Language Interface

12.1 Overview

12.2 Files

12.3 Types

12.3.1 Enumerations

12.3.1.1 `MteJail.Algo` (CS)

12.3.1.1.1 `MteJail.Algo.aNone` (CS)

12.3.1.1.2 `MteJail.Algo.aAndroidArm32Dev` (CS)

12.3.1.1.3 `MteJail.Algo.aAndroidArm64Dev` (CS)

12.3.1.1.4 `MteJail.Algo.aAndroidX86Sim` (CS)

12.3.1.1.5 `MteJail.Algo.aAndroidX86_64Sim` (CS)

12.3.1.1.6 `MteJail.Algo.aIosArm64Dev` (CS)

12.3.1.1.7 `MteJail.Algo.aIosX86_64Sim` (CS)

12.3.2 Classes

12.3.2.1 `MteJail`

12.4 Functions

12.4.1 Jailbreak

12.4.1.1 `MteJail` Constructor (CS)

12.4.1.2 `MteJail.SetAlgo` (CS)

12.4.1.3 `MteJail.SetNonceSeed` (CS)

12.4.1.4 `MteJail.NonceCallback` (CS)

13 Go Language Interface

13.1 Overview

13.2 Files

13.3 Types

13.3.1 Constant Sets

13.3.1.1 `JailAlgo` (Go)

13.3.1.1.1 `JailAlgoNone` (Go)

13.3.1.1.2 `JailAlgoAndroidArm32Dev` (Go)

13.3.1.1.3 `JailAlgoAndroidArm64Dev` (Go)

13.3.1.1.4 `JailAlgoAndroidX86Sim` (Go)

13.3.1.1.5 `JailAlgoAndroidX86_64Sim` (Go)

13.3.1.1.6 `JailAlgoIosArm64Dev` (Go)

13.3.1.1.7 `JailAlgoIosX86_64Sim` (Go)

13.3.2 Structures

13.3.2.1 `Jail`

13.4 Functions

13.4.1 Jailbreak

13.4.1.1 `NewJail` (Go)

13.4.1.2 `(*Jail)SetAlgo` (Go)

13.4.1.3 `(*Jail)SetNonceSeed` (Go)

13.4.1.4 `(*Jail)NonceCallback` (Go)

14 Java Language Interface

14.1 Overview

14.2 Files

14.3 Types

14.3.1 Enumerations
14.3.1.1 `MteJail.Algo` (Java)
14.3.1.1.1 `MteJail.Algo.aNone` (Java)
14.3.1.1.2 `MteJail.Algo.aAndroidArm32Dev` (Java)
14.3.1.1.3 `MteJail.Algo.aAndroidArm64Dev` (Java)
14.3.1.1.4 `MteJail.Algo.aAndroidX86Sim` (Java)
14.3.1.1.5 `MteJail.Algo.aAndroidX86_64Sim` (Java)
14.3.1.1.6 `MteJail.Algo.aIosArm64Dev` (Java)
14.3.1.1.7 `MteJail.Algo.aIosX86_64Sim` (Java)
14.3.2 Classes
14.3.2.1 `MteJail`
14.4 Functions
14.4.1 Jailbreak
14.4.1.1 `MteJail.setAlgo` (Java)
14.4.1.2 `MteJail.setNonceSeed` (Java)
14.4.1.3 `MteJail.nonceCallback` (Java)
15 Objective-C Language Interface
15.1 Overview
15.2 Files
15.3 Types
15.3.1 Enumerations
15.3.1.1 `MteJailAlgo` (ObjC)
15.3.1.1.1 `mteJailAlgoNone` (ObjC)
15.3.1.1.2 `mteJailAlgoAndroidArm32Dev` (ObjC)
15.3.1.1.3 `mteJailAlgoAndroidArm64Dev` (ObjC)
15.3.1.1.4 `mteJailAlgoAndroidX86Sim` (ObjC)
15.3.1.1.5 `mteJailAlgoAndroidX86_64Sim` (ObjC)
15.3.1.1.6 `mteJailAlgoIosArm64Dev` (ObjC)
15.3.1.1.7 `mteJailAlgoIosX86_64Sim` (ObjC)
15.3.2 Classes
15.3.2.1 `MteJail`
15.4 Functions
15.4.1 Jailbreak
15.4.1.1 `MteJail -setAlgo:` (ObjC)
15.4.1.2 `MteJail -setNonceSeed:` (ObjC)
15.4.1.3 `MteJail -nonceCallbackWithMinLength:maxLength:nonce:nBytes:` (ObjC)
16 Python Language Interface
16.1 Overview
16.2 Files
16.3 Types
16.3.1 Enumerations
16.3.1.1 `MteJail.Algo` (Python)
16.3.1.1.1 `MteJail.Algo.a_none` (Python)
16.3.1.1.2 `MteJail.Algo.a_android_arm32_dev` (Python)
16.3.1.1.3 `MteJail.Algo.a_android_arm64_dev` (Python)
16.3.1.1.4 `MteJail.Algo.a_android_x86_sim` (Python)
16.3.1.1.5 `MteJail.Algo.a_android_x86_64_sim` (Python)
16.3.1.1.6 `MteJail.Algo.a_ios_arm64_dev` (Python)

16.3.1.1.7 `MteJail.Algo.a_ios_x86_64_sim` (Python)
16.3.2 Classes
16.3.2.1 `MteJail`
16.4 Functions
16.4.1 Jailbreak
16.4.1.1 `MteJail` Initializer (Python)
16.4.1.2 `MteJail.set_algo` (Python)
16.4.1.3 `MteJail.set_nonce_seed` (Python)
16.4.1.4 `MteJail.nonce_callback` (Python)
17 Swift Language Interface
17.1 Overview
17.2 Files
17.3 Types
17.3.1 Enumerations
17.3.1.1 `MteJail.Algo` (Swift)
17.3.1.1.1 `MteJail.Algo.aNone` (Swift)
17.3.1.1.2 `MteJail.Algo.aAndroidArm32Dev` (Swift)
17.3.1.1.3 `MteJail.Algo.aAndroidArm64Dev` (Swift)
17.3.1.1.4 `MteJail.Algo.aAndroidX86Sim` (Swift)
17.3.1.1.5 `MteJail.Algo.aAndroidX86_64Sim` (Swift)
17.3.1.1.6 `MteJail.Algo.aIosArm64Dev` (Swift)
17.3.1.1.7 `MteJail.Algo.aIosX86_64Sim` (Swift)
17.3.2 Classes
17.3.2.1 `MteJail`
17.4 Functions
17.4.1 Jailbreak
17.4.1.1 `MteJail.setAlgo` (Swift)
17.4.1.2 `MteJail.setNonceSeed` (Swift)
17.4.1.3 `MteJail.nonceCallback` (Swift)
18 WASM Language Interface
18.1 Overview
18.2 Files
18.3 Types
18.3.1 Enumerations
18.3.1.1 `MteJailAlgo` (WASM)
18.3.1.1.1 `MteJailAlgo.aNone` (WASM)
18.3.1.1.2 `MteJailAlgo.aAndroidArm32Dev` (WASM)
18.3.1.1.3 `MteJailAlgo.aAndroidArm64Dev` (WASM)
18.3.1.1.4 `MteJailAlgo.aAndroidX86Sim` (WASM)
18.3.1.1.5 `MteJailAlgo.aAndroidX86_64Sim` (WASM)
18.3.1.1.6 `MteJailAlgo.aIosArm64Dev` (WASM)
18.3.1.1.7 `MteJailAlgo.aIosX86_64Sim` (WASM)
18.3.2 Classes
18.3.2.1 `MteJail`
18.4 Functions
18.4.1 Jailbreak
18.4.1.1 `MteJail` Constructor (WASM)
18.4.1.2 `MteJail.setAlgo` (WASM)

18.4.1.3 `MteJail.setNonceSeed` (WASM)
18.4.1.4 `MteJail.nonceCallback` (WASM)

19 Contact Eclypses

1 Introduction

The Jailbreak Detection Add-On is used in conjunction with the core MTE technology or any add-ons to aid in detecting jailbroken/rooted devices.

A jailbroken (iOS) or rooted (Android) device is a device that has software installed on it not approved by the device vendor (e.g., via an app store). The jailbreak/root process involves purposely removing some security built in to the device's operating system, and therefore makes these devices more susceptible to attacks. App vendors may want to limit or remove the ability of compromised devices to interact with their servers.

The Jailbreak Detection Add-On works by taking a seed nonce and mutating it. The mutations that occur depend on the results of the jailbreak/root checks performed. If a device is communicating to a server (defined as any other endpoint the device is communicating with, which is typically a server), that server will start from the same seed nonce and simulate the same jailbreak/root checks the device is doing. Since the MTE technology depends on paired nodes that are in sync, if the server and device mutate the nonce the same way, they will be able to communicate. However, if any jailbreak/root check indicates the device is jailbroken/rooted, the device's nonce mutation will end up different, and therefore the device and server will not be in sync. When the device and server are not in sync, they will fail to communicate successfully (with extremely high probability if the token size is sufficiently large), which allows the *server* to detect that the *device* is jailbroken/rooted.

Most jailbreak/root detection schemes are written to work solely on the device, which makes them susceptible to counter-attacks where the jailbreak/root system can modify the results or simply bypass the check and the app can't tell. The Jailbreak Detection Add-On offloads the actual detection of the jailbreak/root to the server, so the device can't simply "return false" to a jailbreak/root check function. Note that nonce mutation occurs whether the device is jailbroken/rooted or not, so it cannot just be skipped. The mutation is just different depending on the results of the checks performed.

It is up to the app developer to decide how to handle the result of the detection. The Jailbreak Detection Add-On does not do anything to the app, and the MTE code running on the device doesn't even know that anything is or isn't wrong; it simply does the checks and modifies the nonce.

As with all MTE technology, a paired encoder and decoder need to be in sync. The entropy, nonce, and personalization string must all be the same on both sides. Since the Jailbreak Detection Add-On mutates a seed nonce, that mutation must be done the same way on the encoder (device) and decoder (server). The encoder (device) mutator must correspond to the type of device being checked. Each type of device (or simulator) has its own jailbreak/root detection logic which is not compatible with any other device (or simulator) type since the checks performed are highly specific to the hardware and operating system. The decoder (server) mutator must match the type of device (or simulator) being communicated with. Most systems will have servers that communicate with multiple types of devices (and simulators), so you must have a way of knowing which type you are communicating with and pair them appropriately.

Example: iPhone (ARM64) encoder communicating with Linux (x86_64) decoder: Both should use the iOS ARM64 device mutator.
Example: iPhone Simulator (x86_64) encoder communicating with Linux (x86_64) decoder: Both should use the iOS x86_64 simulator mutator.

On an iOS or Android device, if you choose a mutator that does not match the actual platform, it will fail to communicate as if it is jailbroken. Those devices cannot do simulation of the mutation like other platforms can; they can only do the actual checks that correspond to their hardware and operating system.

The actual detection of a problem occurs in the decode operation on the server, and is detected in an implied way by the decoder returning a "token does not exist" error when attempting to decode. This is a result of the decoder not being in sync with the encoder due to different nonce mutation. This error is general in the sense that it can imply other problems, so it is assumed that your jailbreak detection system has already verified valid communication before doing the jailbreak detection to elimante the other possible reasons.

One possible approach is to first instantiate the encoder and decoder with the entropy, nonce seed, and personalization string without the jailbreak/root detection, send a message and verify correct decoding. If that is successful, redo the same process with the same inputs, but with jailbreak/root detection enabled and verify correct decoding again. If it now gives the "token does not exist" error, the jailbreak/root detection has identified a problem.

The jailbreak/root nonce mutation is implemented in a set of C functions (one for each supported hardware/operating system combination), and each language interface provides an easier way to use them in the respective language via the nonce callback plugin architecture.

2 Terms and Abbreviations

Refer to the *MTE Core Developer's Guide* terms and abbreviations.

3 Prerequisites

Refer to the *MTE Core Developer's Guide* prerequisites.

4 Caveats

Refer to the *MTE Core Developer's Guide* caveats.

5 Architecture

Refer to the *MTE Core Developer's Guide* architecture.

6 SDK Structure

Refer to the *MTE Core Developer's Guide* SDK structure.

All SDK packages that include the MTE Jailbreak Detection Add-On will include the MTE Jailbreak Detection Add-On library and the associated MTE Jailbreak Detection Add-On language interfaces will also be present.

7 Options

Refer to the *MTE Core Developer's Guide* options.

The Jailbreak Detection Add-On requires the ability to mutate a seed nonce, so only DRBGs that accept a nonce can be used. In particular, the `NODF` DRBGs are not usable because they do not accept a nonce.

8 Status

Refer to the *MTE Core Developer's Guide* status.

9 Basic C API

9.1 Overview

The basic flow is to use the appropriate nonce mutator function to mutate the nonce in your nonce callback. Most of the arguments to the nonce mutator are simply passed through from the nonce callback arguments. In addition, the nonce seed must be supplied to the nonce mutator. The result will be the mutated nonce which ends up as the nonce that will be used in the instantiation process which triggered the nonce callback.

9.2 Files

The MTE Jailbreak Detection Add-On uses the core header files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
mte_jail.h	This defines the nonce mutator functions.

9.3 Functions

9.3.1 Nonce Mutator

Nonce mutation functions are declared in `mte_jail.h`.

9.3.1.1 `mte_jail_n_cb_android_arm32_d` (C)

```
void mte_jail_n_cb_android_arm32_d(const void *seed, uint32_t seed_bytes, uint32_t
min_length, uint32_t max_length, void *nonce, uint32_t *n_bytes)
```

Jailbreak detection nonce mutator for an Android ARM32 device. The `min_length`, `max_length`, `nonce`, and `n_bytes` should be passed through from the nonce callback. The `seed` and `seed_bytes` are used to provide your seed nonce which will be mutated during jailbreak detection and the mutated nonce is set in `nonce` and `n_bytes`.

`seed`: the nonce seed value.
`seed_bytes`: the size of the nonce seed (in bytes).
`min_length`: the minimum length of the nonce array (in bytes).
`max_length`: the maximum length of the nonce array (in bytes).
`nonce`: pointer to the nonce array provided by the caller. The mutated nonce will be copied in to this array.
`n_bytes`: the nonce size (in bytes). This will be set to the length of the mutated nonce.

9.3.1.2 `mte_jail_n_cb_android_arm64_d` (C)

```c
void mte_jail_n_cb_android_arm64_d(const void *seed, uint32_t seed_bytes, uint32_t min_length, uint32_t max_length, void *nonce, uint32_t *n_bytes)
```

Jailbreak detection nonce mutator for an Android ARM64 device. The `min_length`, `max_length`, `nonce`, and `n_bytes` should be passed through from the nonce callback. The `seed` and `seed_bytes` are used to provide your seed nonce which will be mutated during jailbreak detection and the mutated nonce is set in `nonce` and `n_bytes`.

`seed`: the nonce seed value.
`seed_bytes`: the size of the nonce seed (in bytes).
`min_length`: the minimum length of the nonce array (in bytes).
`max_length`: the maximum length of the nonce array (in bytes).
`nonce`: pointer to the nonce array provided by the caller. The mutated nonce will be copied in to this array.
`n_bytes`: the nonce size (in bytes). This will be set to the length of the mutated nonce.

9.3.1.3 `mte_jail_n_cb_android_x86_s` (C)

```c
void mte_jail_n_cb_android_x86_s(const void *seed, uint32_t seed_bytes, uint32_t min_length, uint32_t max_length, void *nonce, uint32_t *n_bytes)
```

Jailbreak detection nonce mutator for an Android x86 simulator. The `min_length`, `max_length`, `nonce`, and `n_bytes` should be passed through from the nonce callback. The `seed` and `seed_bytes` are used to provide your seed nonce which will be mutated during jailbreak detection and the mutated nonce is set in `nonce` and `n_bytes`.

`seed`: the nonce seed value.
`seed_bytes`: the size of the nonce seed (in bytes).
`min_length`: the minimum length of the nonce array (in bytes).
`max_length`: the maximum length of the nonce array (in bytes).
`nonce`: pointer to the nonce array provided by the caller. The mutated nonce will be copied in to this array.
`n_bytes`: the nonce size (in bytes). This will be set to the length of the mutated nonce.

9.3.1.4 `mte_jail_n_cb_android_x86_64_s` (C)

```c
void mte_jail_n_cb_android_x86_64_s(const void *seed, uint32_t seed_bytes, uint32_t min_length, uint32_t max_length, void *nonce, uint32_t *n_bytes)
```

Jailbreak detection nonce mutator for an Android x86_64 simulator. The `min_length`, `max_length`, `nonce`, and `n_bytes` should be passed through from the nonce callback. The `seed` and `seed_bytes` are used to provide your seed nonce which will be mutated during jailbreak detection and the mutated nonce is set in `nonce` and `n_bytes`.

`seed`: the nonce seed value.
`seed_bytes`: the size of the nonce seed (in bytes).
`min_length`: the minimum length of the nonce array (in bytes).
`max_length`: the maximum length of the nonce array (in bytes).
`nonce`: pointer to the nonce array provided by the caller. The mutated nonce will be copied in to this array.
`n_bytes`: the nonce size (in bytes). This will be set to the length of the mutated nonce.

9.3.1.5 `mte_jail_n_cb_ios_arm64_d` (C)

```
void mte_jail_n_cb_ios_arm64_d(const void *seed, uint32_t seed_bytes, uint32_t
min_length, uint32_t max_length, void *nonce, uint32_t *n_bytes)
```

Jailbreak detection nonce mutator for an iOS ARM64 device. The `min_length`, `max_length`, `nonce`, and `n_bytes` should be passed through from the nonce callback. The `seed` and `seed_bytes` are used to provide your seed nonce which will be mutated during jailbreak detection and the mutated nonce is set in `nonce` and `n_bytes`.

`seed`: the nonce seed value.
`seed_bytes`: the size of the nonce seed (in bytes).
`min_length`: the minimum length of the nonce array (in bytes).
`max_length`: the maximum length of the nonce array (in bytes).
`nonce`: pointer to the nonce array provided by the caller. The mutated nonce will be copied in to this array.
`n_bytes`: the nonce size (in bytes). This will be set to the length of the mutated nonce.

9.3.1.6 `mte_jail_n_cb_ios_x86_64_s` (C)

```
void mte_jail_n_cb_ios_x86_64_s(const void *seed, uint32_t seed_bytes, uint32_t
min_length, uint32_t max_length, void *nonce, uint32_t *n_bytes)
```

Jailbreak detection nonce mutator for an iOS x86_64 simulator. The `min_length`, `max_length`, `nonce`, and `n_bytes` should be passed through from the nonce callback. The `seed` and `seed_bytes` are used to provide your seed nonce which will be mutated during jailbreak detection and the mutated nonce is set in `nonce` and `n_bytes`.

`seed`: the nonce seed value.
`seed_bytes`: the size of the nonce seed (in bytes).
`min_length`: the minimum length of the nonce array (in bytes).
`max_length`: the maximum length of the nonce array (in bytes).
`nonce`: pointer to the nonce array provided by the caller. The mutated nonce will be copied in to this array.
`n_bytes`: the nonce size (in bytes). This will be set to the length of the mutated nonce.

10 C Support API

The C Support API uses the Basic C API to support jailbreak detection. There is no memory allocation associated with the jailbreak/root detection, so no additional C Support API functions are necessary to support it.

11 C++ Language Interface

11.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, call `setAlgo()` to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Call `setNonceSeed()` to set the nonce seed before calling `instantiate()` on the encoder or decoder.

11.2 Files

The MTE Jailbreak Detection Add-On uses the core header files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.h	The MteJail class declaration.

The MTE Jailbreak Detection Add-On uses the core source files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.cpp	The MteJail class definition.

11.3 Types

11.3.1 Enumerations

11.3.1.1 `MteJail::Algo` (CPP)

Defined in `MteJail.h`.

11.3.1.1.1 `MteJail::Algo::aNone` (CPP)

No choice made. No nonce mutation will be performed.

11.3.1.1.2 `MteJail::Algo::aAndroidArm32Dev` (CPP)

Android ARM32 device.

11.3.1.1.3 `MteJail::Algo::aAndroidArm64Dev` (CPP)

Android ARM64 device.

11.3.1.1.4 `MteJail::Algo::aAndroidX86Sim` (CPP)

Android x86 simulator.

11.3.1.1.5 `MteJail::Algo::aAndroidX86_64Sim` (CPP)

Android x86_64 simulator.

11.3.1.1.6 `MteJail::Algo::aIosArm64Dev` (CPP)

iOS ARM64 device.

11.3.1.1.7 `MteJail::Algo::aIosX86_64Sim` (CPP)

iOS x86_64 simulator.

11.3.2 Classes

11.3.2.1 `MteJail`

Provides jailbreak detection support.

11.4 Functions

11.4.1 Jailbreak

Jailbreak functions are declared in `MteJail.h`.

11.4.1.1 `MteJail` Constructor (CPP)

```
MteJail()
```

Constructor.

11.4.1.2 `MteJail` Destructor (CPP)

```
virtual ~MteJail()
```

Destructor.

11.4.1.3 `MteJail::setAlgo` (CPP)

```
void setAlgo(MteJail::Algo algo)
```

Set the jailbreak detection algorithm to pair with. Defaults to `MteJail::aNone`.

`algo`: the jailbreak detection algorithm.

11.4.1.4 `MteJail::setNonceSeed` (CPP)

```
void setNonceSeed(uint64_t seed)
```

Set the nonce seed.

`seed`: the nonce seed.

11.4.1.5 `MteJail::nonceCallback` (CPP)

```
virtual void nonceCallback(uint32_t minLength, uint32_t maxLength, void *nonce,
uint32_t& nBytes)
```

The nonce callback. See `MteBase::NonceCallback` for details.

12 C# Language Interface

12.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, call `SetAlgo()` to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Call `SetNonceSeed()` to set the nonce seed before calling `Instantiate()` on the encoder or decoder.

12.2 Files

The MTE Jailbreak Detection Add-On uses the core source files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteDelegatesJail.cs	The delegates for MteJail.
MteInteropJail.cs	The MteInterop partial class for MteJail.
MteJail.cs	The MteJail class.

12.3 Types

12.3.1 Enumerations

12.3.1.1 `MteJail.Algo` (CS)

Defined in `MteJail.cs`.

12.3.1.1.1 `MteJail.Algo.aNone` (CS)

No choice made. No nonce mutation will be performed.

12.3.1.1.2 `MteJail.Algo.aAndroidArm32Dev` (CS)

Android ARM32 device.

12.3.1.1.3 `MteJail.Algo.aAndroidArm64Dev` (CS)

Android ARM64 device.

12.3.1.1.4 `MteJail.Algo.aAndroidX86Sim` (CS)

Android x86 simulator.

12.3.1.1.5 `MteJail.Algo.aAndroidX86_64Sim` (CS)

Android x86_64 simulator.

12.3.1.1.6 `MteJail.Algo.aIosArm64Dev` (CS)

iOS ARM64 device.

12.3.1.1.7 `MteJail.Algo.aIosX86_64Sim` (CS)

iOS x86_64 simulator.

12.3.2 Classes

12.3.2.1 `MteJail`

Provides jailbreak detection support.

12.4 Functions

12.4.1 Jailbreak

Jailbreak functions are declared in `MteJail.cs`.

12.4.1.1 `MteJail` Constructor (CS)

```
public MteJail(MteInterop interop)
```

Constructor taking a provided interop.

`interop`: the interop object to use.

12.4.1.2 `MteJail.SetAlgo` (CS)

```
public void SetAlgo(MteJail.Algo algo)
```

Set the jailbreak detection algorithm to pair with. Defaults to `MteJail.Algo.aNone`.

`algo`: the jailbreak detection algorithm.

12.4.1.3 `MteJail.SetNonceSeed` (CS)

```
void SetNonceSeed(UInt64 seed)
```

Set the nonce seed.

`seed`: the nonce seed.

12.4.1.4 `MteJail.NonceCallback` (CS)

```
public virtual void NonceCallback(int minLength, int maxLength, byte[] nonce, out
int nBytes)
```

The nonce callback. See `MteNonceCallback` for details.

13 Go Language Interface

13.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, call `SetAlgo()` to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Call `SetNonceSeed()` to set the nonce seed before calling `Instantiate()` on the encoder or decoder.

13.2 Files

The MTE Jailbreak Detection Add-On uses the core source files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.go	The MteJail structure and functions.

13.3 Types

13.3.1 Constant Sets

13.3.1.1 `JailAlgo` (Go)

Defined in `MteJail.go`.

13.3.1.1.1 `JailAlgoNone` (Go)

No choice made. No nonce mutation will be performed.

13.3.1.1.2 `JailAlgoAndroidArm32Dev` (Go)

Android ARM32 device.

13.3.1.1.3 `JailAlgoAndroidArm64Dev` (Go)

Android ARM64 device.

13.3.1.1.4 `JailAlgoAndroidX86Sim` (Go)

Android x86 simulator.

13.3.1.1.5 `JailAlgoAndroidX86_64Sim` (Go)

Android x86_64 simulator.

13.3.1.1.6 `JailAlgoIosArm64Dev` (Go)

iOS ARM64 device.

13.3.1.1.7 `JailAlgoIosX86_64Sim` (Go)

iOS x86_64 simulator.

13.3.2 Structures

13.3.2.1 `Jail`

Provides jailbreak detection support.

13.4 Functions

13.4.1 Jailbreak

Jailbreak functions are declared in `MteJail.go`.

13.4.1.1 `NewJail` (Go)

```
func NewJail() *Jail
```

Creates a new `MkeJail` structure.

13.4.1.2 `(*Jail)SetAlgo` (Go)

```
func (jail *Jail) SetAlgo(algo JailAlgo)
```

Set the jailbreak detection algorithm to pair with. Defaults to `JailAlgoNone`.

`algo`: the jailbreak detection algorithm.

13.4.1.3 `(*Jail)SetNonceSeed` (Go)

```go
func (jail *Jail) SetNonceSeed(seed uint64)
```

Set the nonce seed.

`seed`: the nonce seed.

13.4.1.4 `(*Jail)NonceCallback` (Go)

```go
func (jail *Jail) NonceCallback(minLength int, maxLength int, nonce []byte, nBytes
*int)
```

The nonce callback. See `MteBase::NonceCallback` for details.

14 Java Language Interface

14.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, call `setAlgo()` to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Call `setNonceSeed()` to set the nonce seed before calling `instantiate()` on the encoder or decoder.

14.2 Files

The MTE Jailbreak Detection Add-On uses the core source files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.java	The MteJail class.

14.3 Types

14.3.1 Enumerations

14.3.1.1 `MteJail.Algo` (Java)

Defined in `MteJail.java`.

14.3.1.1.1 `MteJail.Algo.aNone` (Java)

No choice made. No nonce mutation will be performed.

14.3.1.1.2 `MteJail.Algo.aAndroidArm32Dev` (Java)

Android ARM32 device.

14.3.1.1.3 `MteJail.Algo.aAndroidArm64Dev` (Java)

Android ARM64 device.

14.3.1.1.4 `MteJail.Algo.aAndroidX86Sim` (Java)

Android x86 simulator.

14.3.1.1.5 `MteJail.Algo.aAndroidX86_64Sim` (Java)

Android x86_64 simulator.

14.3.1.1.6 `MteJail.Algo.aIosArm64Dev` (Java)

iOS ARM64 device.

14.3.1.1.7 `MteJail.Algo.aIosX86_64Sim` (Java)

iOS x86_64 simulator.

14.3.2 Classes

14.3.2.1 `MteJail`

Provides jailbreak detection support.

14.4 Functions

14.4.1 Jailbreak

Jailbreak functions are declared in `MteJail.java`.

14.4.1.1 `MteJail.setAlgo` (Java)

```
public void setAlgo(MteJail.Algo algo)
```

Set the jailbreak detection algorithm to pair with. Defaults to `MteJail.Algo.aNone`.

`algo`: the jailbreak detection algorithm.

14.4.1.2 `MteJail.setNonceSeed` (Java)

```
void setNonceSeed(long seed)
```

Set the nonce seed.

`seed`: the nonce seed.

14.4.1.3 `MteJail.nonceCallback` (Java)

```
public int nonceCallback(int minLength, int maxLength, ByteBuffer nonce)
```

The nonce callback. See `MteBase.NonceCallback` for details.

15 Objective-C Language Interface

15.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, send the `-setAlgo:` message to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Send the `-setNonceSeed:` message to set the nonce seed before sending `-instantiate:` to the encoder or decoder.

15.2 Files

The MTE Jailbreak Detection Add-On uses the core header files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.h	The MteJail class declaration.

The MTE Jailbreak Detection Add-On uses the core source files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.m	The MteJail class definition.

15.3 Types

15.3.1 Enumerations

15.3.1.1 `MteJailAlgo` (ObjC)

Defined in `MteJail.h`.

15.3.1.1.1 `mteJailAlgoNone` (ObjC)

No choice made. No nonce mutation will be performed.

15.3.1.1.2 `mteJailAlgoAndroidArm32Dev` *(ObjC)*

Android ARM32 device.

15.3.1.1.3 `mteJailAlgoAndroidArm64Dev` *(ObjC)*

Android ARM64 device.

15.3.1.1.4 `mteJailAlgoAndroidX86Sim` *(ObjC)*

Android x86 simulator.

15.3.1.1.5 `mteJailAlgoAndroidX86_64Sim` *(ObjC)*

Android x86_64 simulator.

15.3.1.1.6 `mteJailAlgoIosArm64Dev` *(ObjC)*

iOS ARM64 device.

15.3.1.1.7 `mteJailAlgoIosX86_64Sim` *(ObjC)*

iOS x86_64 simulator.

15.3.2 Classes

15.3.2.1 `MteJail`

Provides jailbreak detection support.

15.4 Functions

15.4.1 Jailbreak

Jailbreak functions are declared in `MteJail.h`.

15.4.1.1 `MteJail -setAlgo:` *(ObjC)*

```
-(void)setAlgo:(MteJailAlgo)algo
```

Set the jailbreak detection algorithm to pair with. Defaults to `MteJail.Algo.aNone`.

`algo`: the jailbreak detection algorithm.

15.4.1.2 `MteJail -setNonceSeed:` (ObjC)

```
-(void)setNonceSeed:(uint64_t)seed
```

Set the nonce seed.

`seed`: the nonce seed.

15.4.1.3 `MteJail -nonceCallbackWithMinLength:maxLength:nonce:nBytes:` (ObjC)

```
-(void)nonceCallbackWithMinLength:(uint32_t)minLength
                        maxLength:(uint32_t)maxLength
                            nonce:(void *)nonce
                           nBytes:(uint32_t *)nBytes
```

The nonce callback. See `MteNonceCallback` for details.

16 Python Language Interface

16.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, call `set_algo()` to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Call `set_nonce_seed()` to set the nonce seed before calling `instantiate()` on the encoder or decoder.

16.2 Files

The MTE Jailbreak Detection Add-On uses the core source files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.py	The MteJail class.

16.3 Types

16.3.1 Enumerations

16.3.1.1 `MteJail.Algo` (Python)

Defined in `MteJail.py`.

16.3.1.1.1 `MteJail.Algo.a_none` (Python)

No choice made. No nonce mutation will be performed.

16.3.1.1.2 `MteJail.Algo.a_android_arm32_dev` (Python)

Android ARM32 device.

16.3.1.1.3 `MteJail.Algo.a_android_arm64_dev` (Python)

Android ARM64 device.

16.3.1.1.4 `MteJail.Algo.a_android_x86_sim` (Python)

Android x86 simulator.

16.3.1.1.5 `MteJail.Algo.a_android_x86_64_sim` (Python)

Android x86_64 simulator.

16.3.1.1.6 `MteJail.Algo.a_ios_arm64_dev` (Python)

iOS ARM64 device.

16.3.1.1.7 `MteJail.Algo.a_ios_x86_64_sim` (Python)

iOS x86_64 simulator.

16.3.2 Classes

16.3.2.1 `MteJail`

Provides jailbreak detection support.

16.4 Functions

16.4.1 Jailbreak

Jailbreak functions are declared in `MteJail.py`.

16.4.1.1 `MteJail` Initializer (Python)

```
def __init__(self)
```

Initializer.

16.4.1.2 `MteJail.set_algo` (Python)

```
def set_algo(self, algo: MteJail.Algo) -> None
```

Set the jailbreak detection algorithm to pair with. Defaults to `MteJail.Algo.a_none`.

`algo`: the jailbreak detection algorithm.

16.4.1.3 `MteJail.set_nonce_seed` (Python)

```
def set_nonce_seed(self, seed: int) -> None
```

Set the nonce seed.

`seed`: the nonce seed.

16.4.1.4 `MteJail.nonce_callback` (Python)

```
def nonce_callback(self, min_length: int, max_length: int, nonce: bytearray,
n_bytes: List[int]) -> None
```

The nonce callback. See `MteNonceCallback` for details.

17 Swift Language Interface

17.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, call `setAlgo()` to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Call `setNonceSeed()` to set the nonce seed before calling `instantiate()` on the encoder or decoder.

17.2 Files

The MTE Jailbreak Detection Add-On uses the core source files documented in the *MTE Core Developer's Guide*, as well as the following:

File	Description
MteJail.swift	The MteJail class.

17.3 Types

17.3.1 Enumerations

17.3.1.1 `MteJail.Algo` (Swift)

Defined in `MteJail.swift`.

17.3.1.1.1 `MteJail.Algo.aNone` (Swift)

No choice made. No nonce mutation will be performed.

17.3.1.1.2 `MteJail.Algo.aAndroidArm32Dev` (Swift)

Android ARM32 device.

17.3.1.1.3 `MteJail.Algo.aAndroidArm64Dev` (Swift)

Android ARM64 device.

17.3.1.1.4 `MteJail.Algo.aAndroidX86Sim` (Swift)

Android x86 simulator.

17.3.1.1.5 `MteJail.Algo.aAndroidX86_64Sim` (Swift)

Android x86_64 simulator.

17.3.1.1.6 `MteJail.Algo.aIosArm64Dev` (Swift)

iOS ARM64 device.

17.3.1.1.7 `MteJail.Algo.aIosX86_64Sim` (Swift)

iOS x86_64 simulator.

17.3.2 Classes

17.3.2.1 `MteJail`

Provides jailbreak detection support.

17.4 Functions

17.4.1 Jailbreak

Jailbreak functions are declared in `MteJail.swift`.

17.4.1.1 `MteJail.setAlgo` (Swift)

```
public func setAlgo(_ algo: MteJail.Algo)
```

Set the jailbreak detection algorithm to pair with. Defaults to `MteJail.Algo.aIosX86_64Sim` when building for the x86_64 simulator target, `MteJail.Algo.aIosArm64Dev` when building for the ARM64 device target, or `MteJail.Algo.aNone` otherwise.

`algo`: the jailbreak detection algorithm.

17.4.1.2 `MteJail.setNonceSeed` (Swift)

```
public func setNonceSeed(_ seed: UInt64)
```

Set the nonce seed.

`seed`: the nonce seed.

17.4.1.3 `MteJail.nonceCallback` (Swift)

```
public func nonceCallback(_ minLength: Int, _ maxLength: Int, _ nonce: inout
[UInt8], _ nBytes: inout Int)
```

The nonce callback. See `MteNonceCallback` for details.

18 WASM Language Interface

18.1 Overview

The basic flow is to use the nonce mutator class as the nonce callback plugin for the encoder or decoder.

After creating the nonce mutator object, call `setAlgo()` to set the jailbreak detection algorithm to match the device or simulator being used, then set the object as the nonce callback plugin for the encoder or decoder. Call `setNonceSeed()` to set the nonce seed before calling `instantiate()` on the encoder or decoder.

18.2 Files

The MTE Fixed-Length Add-On uses the core source files documented in the *MTE Core Developer's Guide*.

18.3 Types

18.3.1 Enumerations

18.3.1.1 `MteJailAlgo` (WASM)

Defined in `MteJail.h`.

18.3.1.1.1 `MteJailAlgo.aNone` (WASM)

No choice made. No nonce mutation will be performed.

18.3.1.1.2 `MteJailAlgo.aAndroidArm32Dev` (WASM)

Android ARM32 device.

18.3.1.1.3 `MteJailAlgo.aAndroidArm64Dev` (WASM)

Android ARM64 device.

18.3.1.1.4 `MteJailAlgo.aAndroidX86Sim` (WASM)

Android x86 simulator.

18.3.1.1.5 `MteJailAlgo.aAndroidX86_64Sim` (WASM)

Android x86_64 simulator.

18.3.1.1.6 `MteJailAlgo.aIosArm64Dev` (WASM)

iOS ARM64 device.

18.3.1.1.7 `MteJailAlgo.aIosX86_64Sim` (WASM)

iOS x86_64 simulator.

18.3.2 Classes

18.3.2.1 `MteJail`

Provides jailbreak detection support.

18.4 Functions

18.4.1 Jailbreak

18.4.1.1 `MteJail` Constructor (WASM)

```
constructor(base: MteBase)
```

Constructor taking an `MteBase` object or derived encoder/decoder.

`base`: a base or encoder or decoder object.

18.4.1.2 `MteJail.setAlgo` (WASM)

```
setAlgo = (algo: MteJailAlgo): void
```

Set the jailbreak detection algorithm to pair with. Defaults to `MteJailAlgo.aNone`.

`algo`: the jailbreak detection algorithm.

18.4.1.3 `MteJail.setNonceSeed` (WASM)

```
setNonceSeed = (seed: string): void
```

Set the nonce seed.

`seed`: the nonce seed.

18.4.1.4 `MteJail.nonceCallback` (WASM)

```
nonceCallback = (minLength: number, maxLength: number, nonce: number, nBytes:
number): void
```

The nonce callback. See `MteNonceCallback` for details.

19 Contact Eclypses

For more information, please contact:

info@eclypses.com

www.eclypses.com

+1.719.323.6680

All trademarks of Eclypses Inc. may not be used without Eclypses Inc.'s prior written consent. No license for any use thereof has been granted without express written consent. Any unauthorized use thereof may violate copyright laws, trademark laws, privacy and publicity laws and communications regulations and statutes. The names, images and likeness of the Eclypses logo, along with all representations thereof, are valuable intellectual property assets of Eclypses, Inc. Accordingly, no party or parties, without the prior written consent of Eclypses, Inc., (which may be withheld in Eclypses' sole discretion), use or permit the use of any of the Eclypses trademarked names or logos of Eclypses, Inc. for any purpose other than as part of the address for the Premises, or use or permit the use of, for any purpose whatsoever, any image or rendering of, or any design based on, the exterior appearance or profile of the Eclypses trademarks and or logo(s).